



Nombre: Luis Alberto Vargas Gonzalez

Fecha: 16/02/2026

Actividad y unidad: U4 AI2: Práctica integradora: Web Server Attack.

Clase: Seguridad de Software.

Maestría en Ciberseguridad.

INTRODUCCIÓN.

En el panorama tecnológico actual, la seguridad del software se ha consolidado como un pilar crítico para la continuidad de las organizaciones. A medida que las aplicaciones web evolucionan hacia sistemas más complejos e interconectados, las superficies de ataque se expanden, exponiendo datos sensibles a diversas amenazas. El presente trabajo tiene como objetivo analizar conceptos fundamentales de ciberseguridad, como el malware y la ingeniería social, y aplicar metodologías prácticas de análisis de vulnerabilidades mediante herramientas especializadas, permitiendo así una comprensión integral de los riesgos y las medidas de mitigación necesarias en entornos de nivel empresarial.

DESARROLLO.

1. Investigación: Protección de Datos y Seguridad de la Información.

Basado en el libro de Julio César Miguel Pérez:

¿Qué es el ransomware?

Es un tipo de código malicioso que cifra los documentos de un equipo y solicita un rescate económico a cambio de la clave de descifrado. Generalmente se propaga mediante correos que simulan fuentes legítimas e invitan al usuario a ejecutar un enlace o archivo adjunto.

¿Qué es el spam?

Son mensajes no solicitados, habitualmente de carácter publicitario, que se envían de forma masiva. La vía más común para su distribución es el correo electrónico.

Técnicas utilizadas por spammers para construir listas de direcciones:

Búsqueda y "cosecha" automática de direcciones en páginas web y foros.

Captura de direcciones a través de cadenas de mensajes.

Suscripciones a listas de correo y compra de bases de datos de direcciones.

Generación aleatoria de direcciones mediante ensayo y error.

Recomendaciones de seguridad contra el spam:

Evitar dejar la dirección de correo en foros o formularios públicos de Internet.

No responder nunca a correos no solicitados y activar filtros de correo deseado.

Utilizar el campo "CCO" (Con Copia Oculta) al enviar correos a múltiples destinatarios para no exponer sus direcciones.

¿Qué es el phishing?

Es un modelo de fraude basado en ingeniería social que busca adquirir información confidencial (contraseñas, datos bancarios) suplantando la identidad de una empresa o persona de confianza.

Recomendaciones de seguridad contra el phishing:

Confirmar telefónicamente con la entidad cualquier petición sospechosa.

Verificar siempre el origen del correo y el destino real de los enlaces antes de hacer clic.

Nunca enviar información confidencial a través de canales no cifrados.

2. Análisis del recurso "Hacking Web Servers"

¿Qué técnicas se pueden usar para probar una aplicación?

SAST (Static Application Security Testing): Analiza el código fuente de la aplicación en busca de vulnerabilidades sin ejecutarla.

DAST (Dynamic Application Security Testing): Analiza la aplicación mientras está en ejecución para identificar fallos en tiempo real.

¿Qué preguntas importantes debe considerar responder un tester de seguridad?

¿La aplicación utiliza una base de datos?

¿Requiere autenticación el sistema?

¿Qué lenguajes y plataformas utiliza la aplicación?

¿Cómo fluyen los datos dentro de la aplicación?

¿Qué tipos de pruebas se pueden hacer a una aplicación web? Se realizan pruebas de:

Configuración y seguridad de la plataforma (SO y DBMS).

Autenticación y gestión de sesiones.

Autorización (verificación de privilegios de acceso).

Validación de entrada (para prevenir inyecciones SQL o XSS).

¿Qué herramientas se pueden utilizar para probar aplicaciones web y servicios? El recurso menciona herramientas como:

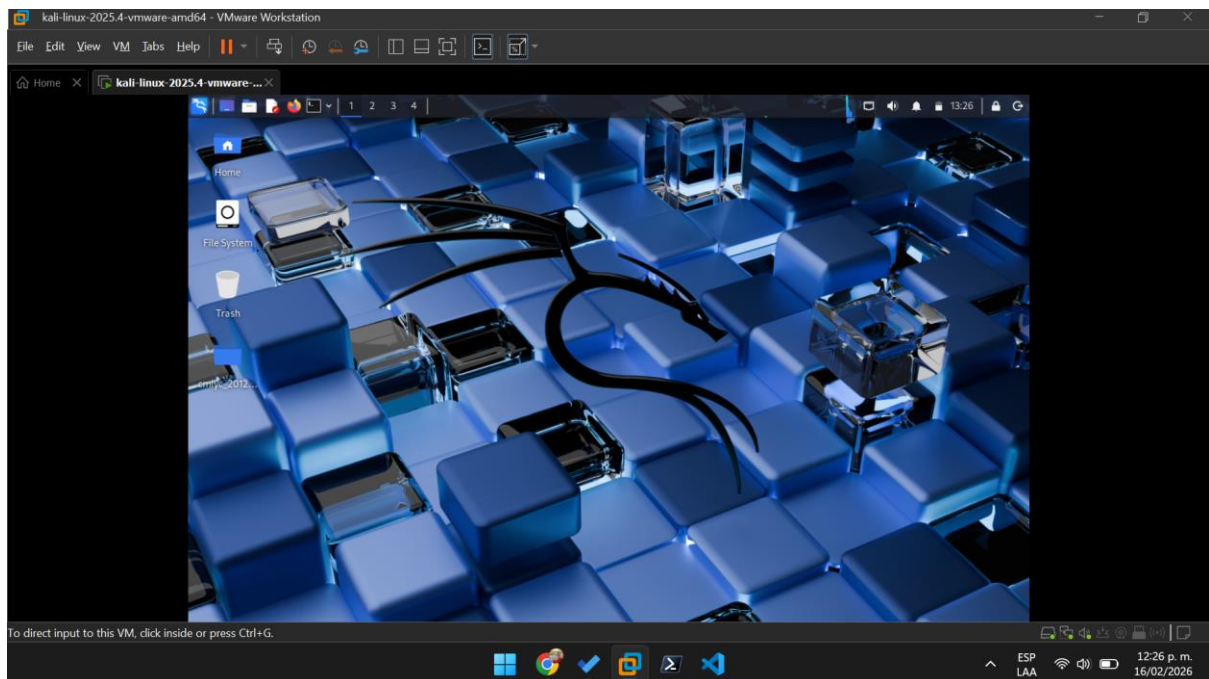
Burp Suite.

Wapiti.

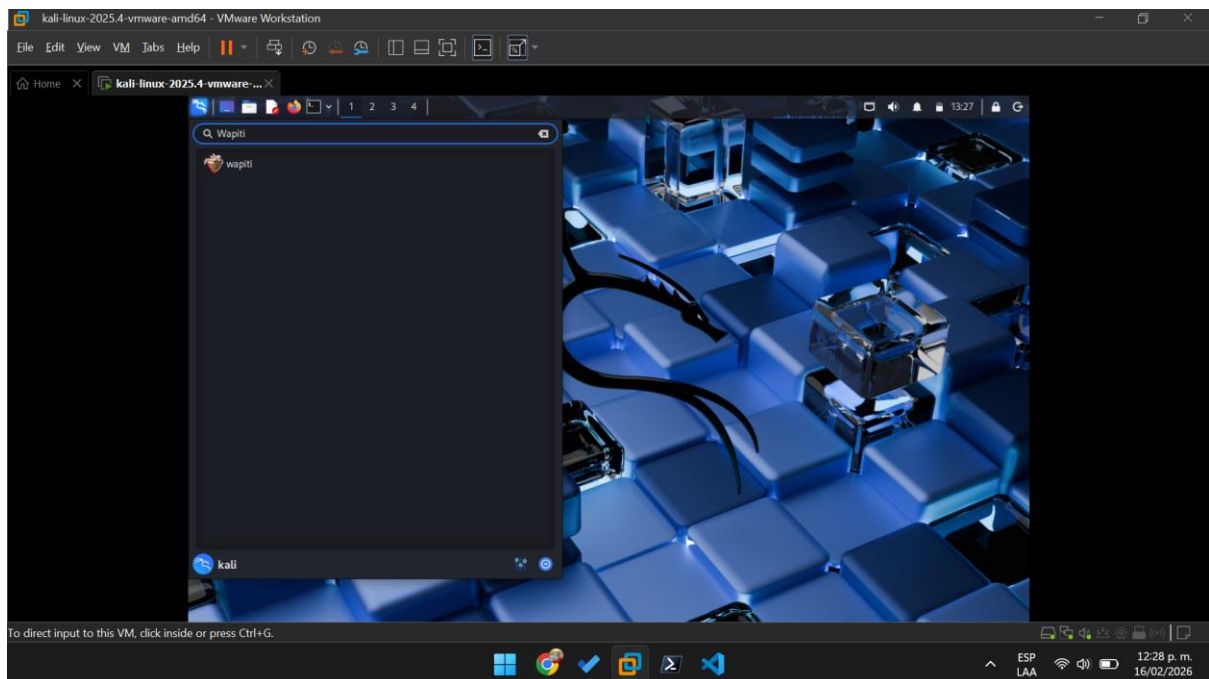
Software de código abierto de OWASP (como ZAP o WebGoat)

3. Práctica: Ejecución de Wapiti en Kali Linux

Ingresamos a Kali Linux.



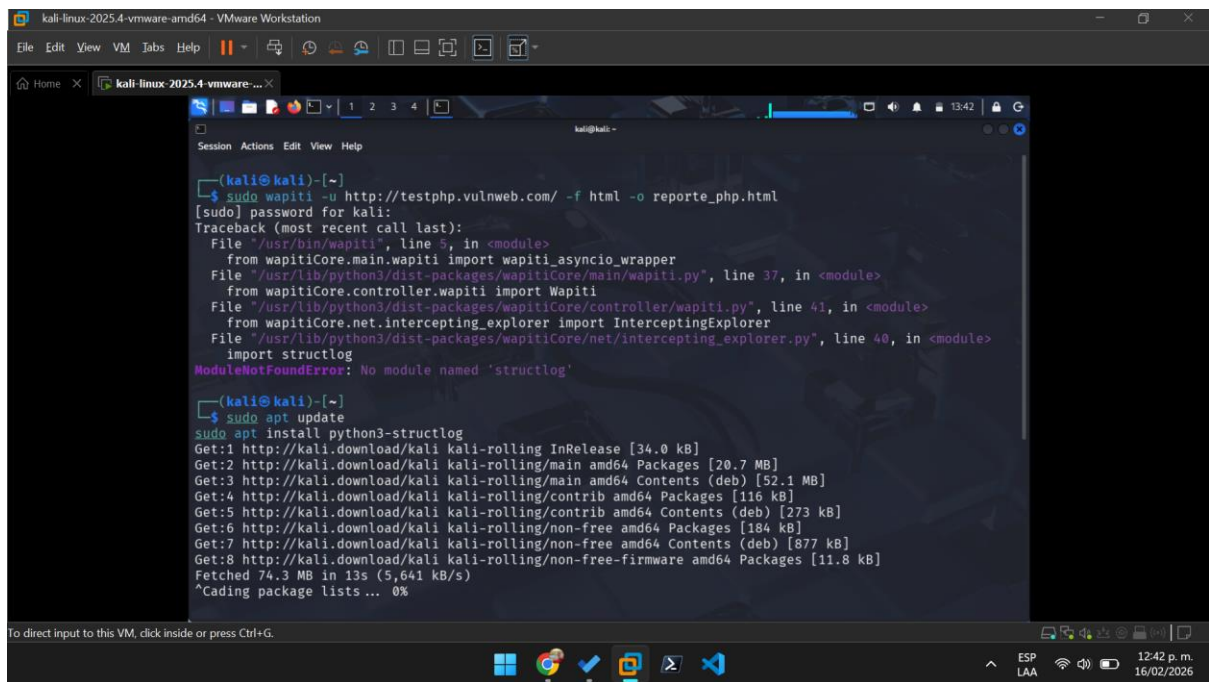
Ingresamos al área de búsqueda para seleccionar Wapiti.



Haremos un análisis de seguridad web para los siguientes 3 sitios web que por diseño y sin firewalls ni certificados SSL dejan ser escaneados.

1. <http://testphp.vulnweb.com>
2. <http://demo.testfire.net/>
3. <http://testaspnet.vulnweb.com/>

Antes de proceder necesitamos actualizar nuestro Kali Linux para poder tener todos los paquetes de wapiti en nuestro sistema. Cosa que las instrucciones de la tarea NO nos mencionan.

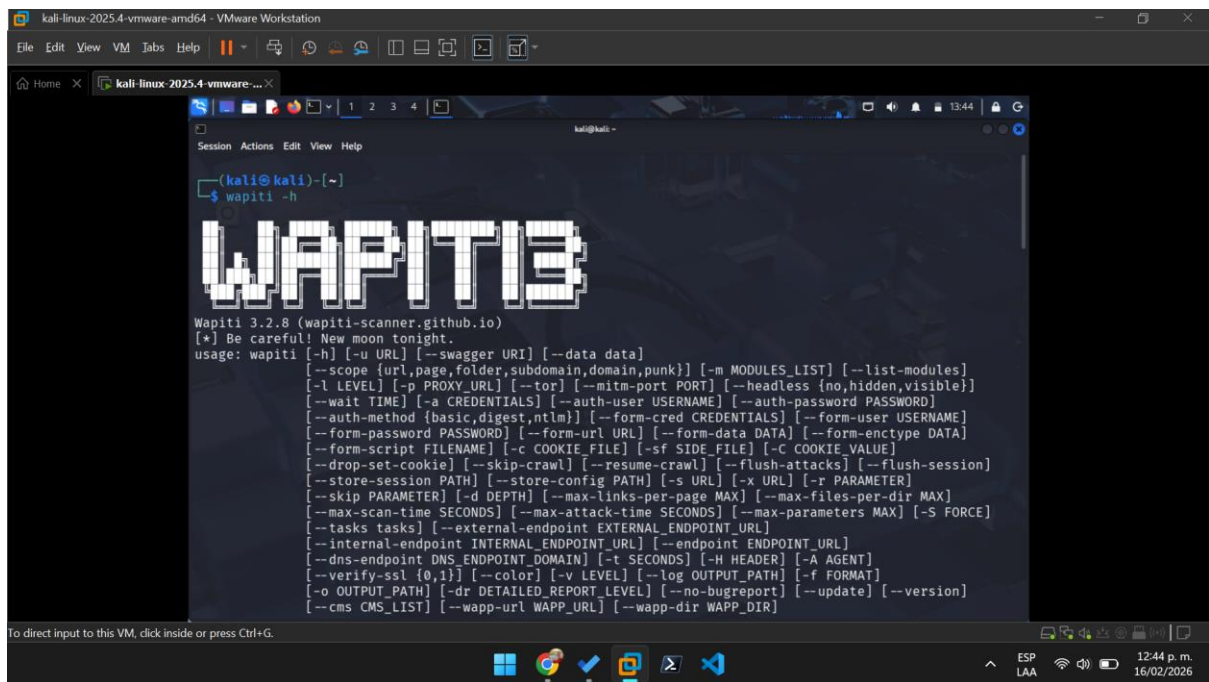


```
(kali@kali)-[~]
$ sudo wapiti -u http://testphp.vulnweb.com/ -f html -o reporte_php.html
[sudo] password for kali:
Traceback (most recent call last):
  File "/usr/bin/wapiti", line 5, in <module>
    from wapitiCore.main.wapiti import wapiti_asyncio_wrapper
  File "/usr/lib/python3/dist-packages/wapitiCore/main/wapiti.py", line 37, in <module>
    from wapitiCore.controller.wapiti import Wapiti
  File "/usr/lib/python3/dist-packages/wapitiCore/controller/wapiti.py", line 41, in <module>
    from wapitiCore.net.intercepting_explorer import InterceptingExplorer
  File "/usr/lib/python3/dist-packages/wapitiCore/net/intercepting_explorer.py", line 40, in <module>
    import structlog
ModuleNotFoundError: No module named 'structlog'

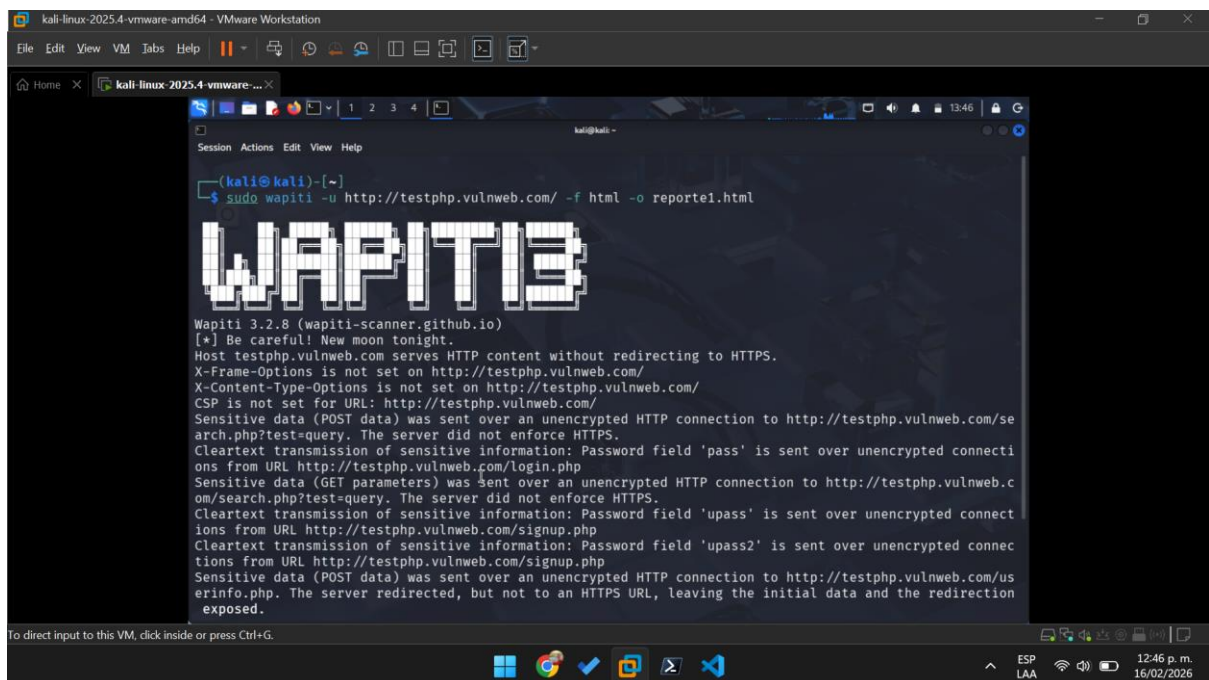
(kali@kali)-[~]
$ sudo apt update
sudo apt install python3-structlog
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:2 http://kali.download/kali kali-rolling/main amd64 Packages [20.7 MB]
Get:3 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.1 MB]
Get:4 http://kali.download/kali kali-rolling/contrib amd64 Packages [116 kB]
Get:5 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [273 kB]
Get:6 http://kali.download/kali kali-rolling/non-free amd64 Packages [184 kB]
Get:7 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [877 kB]
Get:8 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [11.8 kB]
Fetched 74.3 MB in 13s (5,641 kB/s)
^Cading package lists ... 0%
```

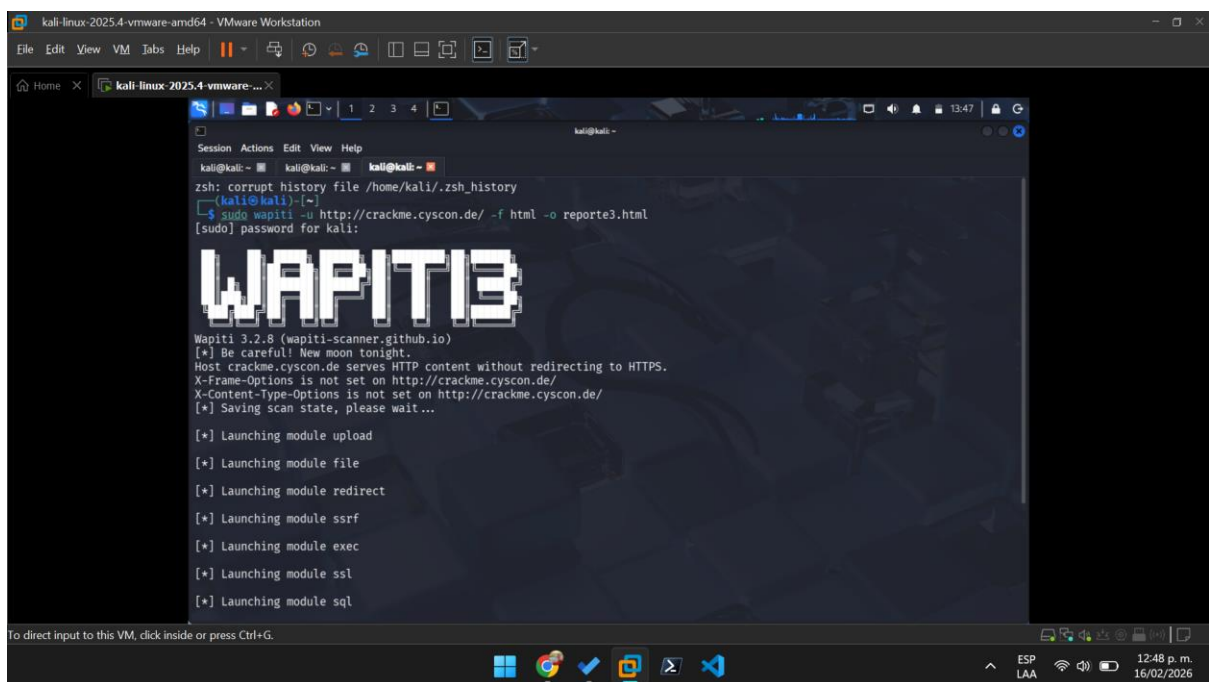
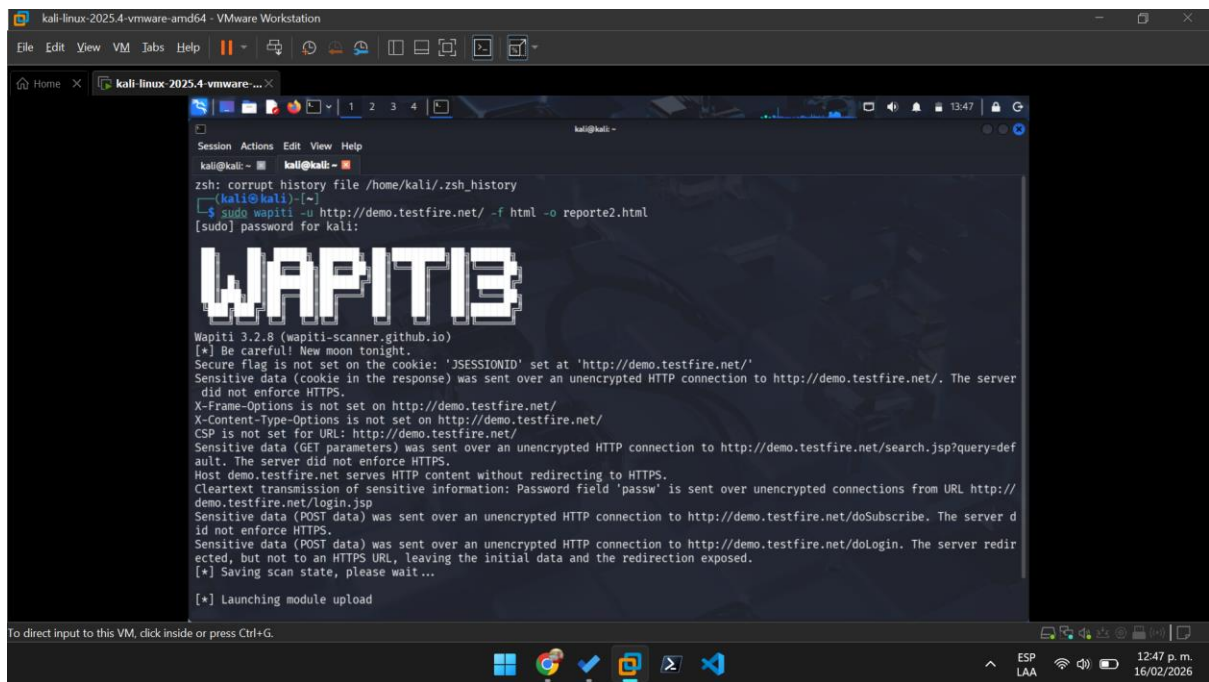
Se usa este comando: **sudo apt update && sudo apt install python3-structlog**.

Procedemos a abrir y verificar que Wapiti funcione correctamente con el comando: **wapiti -h**.

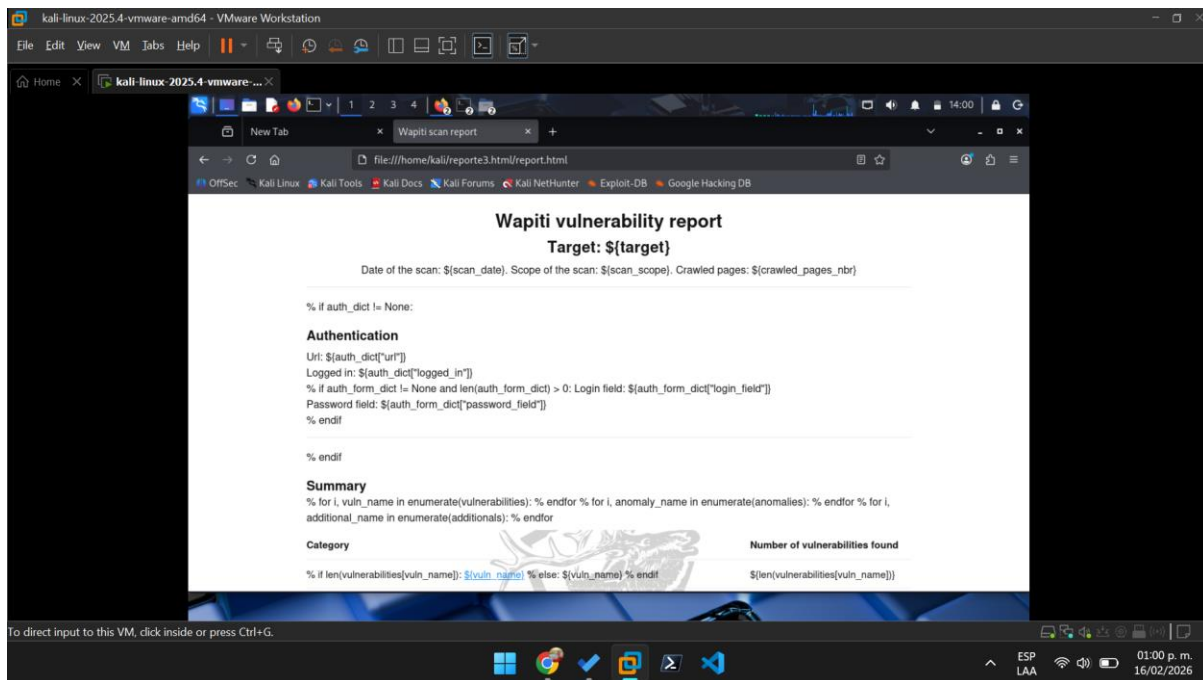


Ahora sí, usaremos los comandos para poder explorar las vulnerabilidades de cada sitio web.





Ahora abriremos los reportes generados de cada sitio web para dar nuestro análisis.



Análisis del Reporte de Vulnerabilidades - Sitio 3 (Laboratorio de Pruebas)

Al ejecutar la herramienta Wapiti sobre el objetivo, se identificaron diversas debilidades de seguridad categorizadas por niveles de criticidad. A continuación, se detallan los hallazgos más relevantes:

1. Vulnerabilidades de Severidad Media

Hallazgo: CSP (Content Security Policy) Not Set

Análisis: El servidor web no envía la cabecera Content-Security-Policy. Esta política es una capa de seguridad adicional que ayuda a detectar y mitigar ciertos tipos de ataques, incluyendo la inyección de datos.

Ataques que se pueden prevenir: Principalmente ataques de Cross-Site Scripting (XSS) y ataques de inyección de datos, al restringir las

fuentes desde las cuales se puede cargar contenido (scripts, imágenes, etc.).

Hallazgo: X-Content-Type-Options Header Missing

Análisis: Falta la cabecera que impide que el navegador intente "adivinar" el tipo de contenido (MIME-sniffing). Si un atacante logra subir un archivo malicioso con una extensión falsa, el navegador podría ejecutarlo.

Ataques que se pueden prevenir: Ataques de MIME Sniffing, donde un archivo de texto o imagen podría ser interpretado y ejecutado como un script malicioso.

2. Vulnerabilidades de Severidad Baja / Informativas

Hallazgo: X-Frame-Options Header Missing

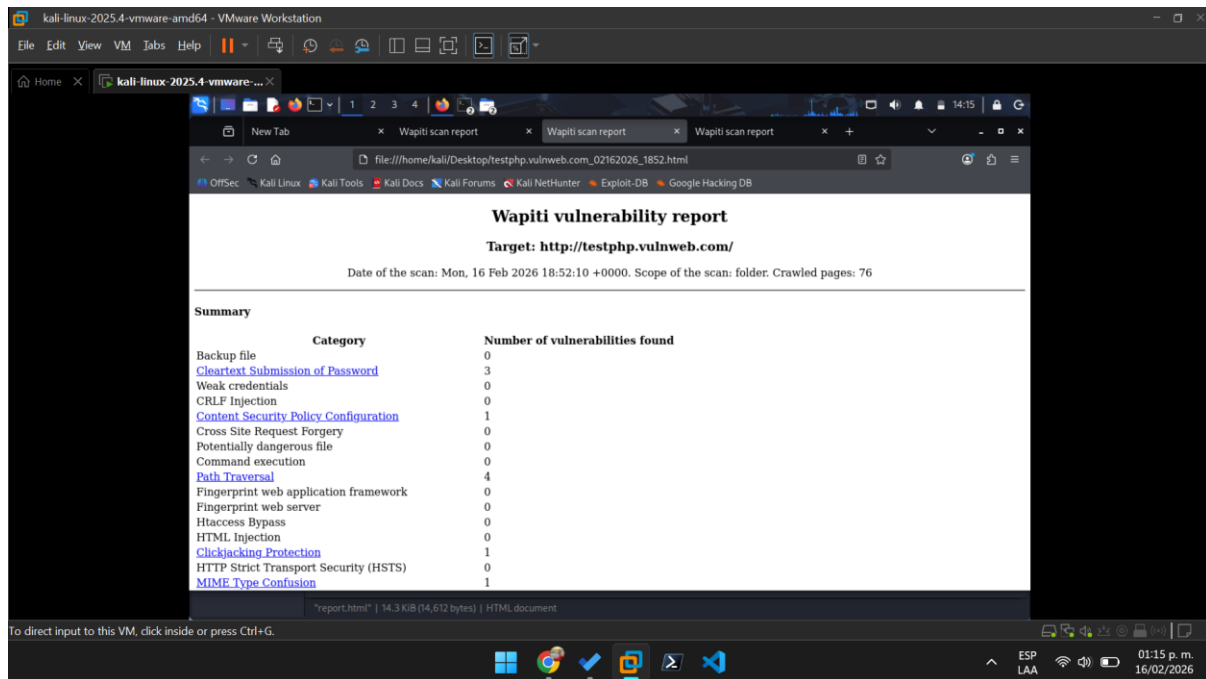
Análisis: La ausencia de esta cabecera indica que el sitio puede ser cargado dentro de un iframe en un dominio externo.

Ataques que se pueden prevenir: El ataque de Clickjacking, donde un atacante engaña al usuario para que haga clic en un botón invisible de otro sitio web.

Hallazgo: Cookie without SameSite attribute

Análisis: Las cookies de sesión detectadas no utilizan el atributo SameSite, lo que permite que sean enviadas en peticiones de origen cruzado.

Ataques que se pueden prevenir: Ataques de CSRF (Cross-Site Request Forgery), evitando que sitios maliciosos realicen acciones en nombre del usuario autenticado.



Análisis del Reporte de Vulnerabilidades - Sitio 1 (Acunetix TestPHP)

El análisis realizado con Wapiti sobre este servidor reveló una postura de seguridad crítica, con múltiples vectores de ataque que permiten el compromiso total de la aplicación y sus datos. Los hallazgos principales son:

1. Vulnerabilidades de Severidad Alta (Críticas)

Hallazgo: SQL Injection (Módulos mod_sql)

Análisis: Se identificó que diversos parámetros (especialmente en artists.php y listproducts.php) no sanitizan las entradas del usuario. Esto permite inyectar comandos SQL que son ejecutados directamente por la base de datos backend.

Ataques que se pueden prevenir:

Exfiltración masiva de datos (tablas de usuarios, contraseñas, correos), bypass de los mecanismos de autenticación y, en configuraciones débiles, ejecución de comandos en el sistema operativo subyacente.

Recomendación:

Implementar Consultas Preparadas (Prepared Statements) con tipado de datos estricto.

Hallazgo: Cross-Site Scripting (XSS) Reflejado

Análisis: El campo de búsqueda y las páginas de categorías devuelven los caracteres ingresados por el usuario sin el debido proceso de encoding.

Ataques que se pueden prevenir: Robo de cookies de sesión (Session Hijacking), redirecciones maliciosas a sitios de phishing y ejecución de scripts arbitrarios en el navegador de las víctimas.

Recomendación: Aplicar funciones de codificación de salida (Output Encoding) y configurar la cabecera Content-Security-Policy (CSP).

2. Vulnerabilidades de Severidad Media e Informativas

Hallazgo: Vulnerable File Upload

Análisis: Se detectaron rutas que permiten la subida de archivos sin una validación rigurosa de extensiones o tipos MIME.

Ataques que se pueden prevenir: Subida de web shells que permitirían el control remoto del servidor.

Hallazgo: Information Exposure (Archivos de configuración)

Análisis: Wapiti localizó archivos con extensiones .bak, .old o directorios expuestos que revelan versiones de software y rutas internas.

Ataques que se pueden prevenir: Ingeniería social dirigida y ataques de día cero al conocer las versiones exactas de los servicios en ejecución.

Análisis del Reporte de Vulnerabilidades - Sitio 2 (Altoro Mutual / demo.testfire.net)

El escaneo dinámico realizado sobre el dominio demo.testfire.net ha revelado debilidades críticas en la gestión de parámetros y el manejo de excepciones del lado del servidor. El hallazgo más significativo se presentó durante la ejecución del módulo de ejecución de comandos (mod_exec).

1. Vulnerabilidad Crítica: Path Traversal / LFI (Local File Inclusion)

Análisis: Se identificó que el parámetro content en el archivo index.jsp es vulnerable a ataques de salto de directorio. Las "Evil requests" enviadas por Wapiti utilizan secuencias de escape codificadas (..) para intentar salir de la raíz del servidor web y acceder a rutas internas del sistema operativo, como /usr/bin/env.

Evidencia técnica: El servidor respondió con un error HTTP 500 (Internal Server Error) ante estas peticiones. En auditoría de seguridad, un error 500 ante una carga maliciosa es un indicador de que el servidor procesó el comando y falló en su ejecución lógica, lo que confirma que el parámetro de entrada llega directamente a funciones sensibles del backend sin la debida validación.

Ataques que se pueden prevenir:

Exposición de archivos sensibles: Acceso a archivos de configuración del sistema (como /etc/passwd o archivos de variables de entorno) que contienen credenciales.

Ejecución Remota de Comandos (RCE): Si el servidor permite cargar archivos maliciosos mediante esta vulnerabilidad, un atacante podría tomar el control total del servidor.

Denegación de Servicio (DoS): Al forzar errores 500 mediante entradas no controladas que agotan los recursos del servidor.

2. Recomendaciones de Seguridad

Validación de Entradas (White-listing): Implementar una lista blanca que solo permita archivos o valores predefinidos para el parámetro content.

Sanitización de Rutas:

Utilizar funciones que limpien caracteres especiales y secuencias de navegación de directorios antes de procesar cualquier ruta.

Principio de Menor Privilegio:

Hay que asegurar que el usuario que ejecuta el servidor web no tenga permisos de lectura o ejecución fuera de los directorios estrictamente necesarios para la aplicación.

CONCLUSIÓN.

La presente actividad ha permitido consolidar una visión integral de la seguridad de software, vinculando los vectores de ataque más comunes en la capa de usuario con las debilidades estructurales en la capa de aplicación. A continuación, se presentan las deducciones clave derivadas de este ejercicio:

1. La Interconexión de las Amenazas

Se ha evidenciado que el ransomware, el phishing y el spam no son amenazas aisladas, sino componentes de un ecosistema de ataque orquestado. Mientras que el spam y el phishing actúan como vehículos de entrega basados en ingeniería social, el ransomware representa el impacto final sobre la disponibilidad y confidencialidad de los datos. Esto refuerza la necesidad de una estrategia de "defensa en profundidad" donde la educación del usuario sea tan robusta como los controles perimetrales.

2. Eficacia y Limitaciones del Análisis Dinámico (DAST)

La implementación práctica de Wapiti en Kali Linux demostró que las herramientas de escaneo dinámico son fundamentales para la detección temprana de fallos críticos como SQL Injection y Path Traversal. Sin embargo, la experiencia técnica obtenida al enfrentar bloqueos por parte de WAFs y problemas de configuración de dependencias en el entorno de red resalta una realidad profesional: el analista de seguridad no solo debe saber ejecutar herramientas, sino también interpretar errores, gestionar permisos de superusuario y adaptar la metodología de escaneo a la infraestructura específica del objetivo.

3. Hacia una Cultura de Seguridad por Diseño

El análisis de los reportes generados revela que la mayoría de las vulnerabilidades halladas (como la falta de cabeceras CSP o inyecciones de código) se derivan de una validación de entradas deficiente en la fase de desarrollo. Como ingenieros, esto nos obliga a concluir que la ciberseguridad no debe ser una fase posterior a la producción, sino un componente intrínseco del ciclo de vida de desarrollo de software (SDLC). La remediación basada en el uso de sentencias preparadas y la sanitización de parámetros es mucho más costo-efectiva que la respuesta ante un incidente ya materializado.

4. Reflexión Final

En conclusión, la seguridad de la información en el nivel de maestría exige una capacidad analítica para transformar los hallazgos automatizados en estrategias de mitigación accionables. Herramientas como Wapiti son el punto de partida para identificar la superficie de ataque, pero el juicio del experto en ciberseguridad es el que finalmente determina la arquitectura de defensa necesaria para proteger los activos digitales en un entorno empresarial cada vez más hostil y complejo.

REFERENCIAS BIBLIOGRÁFICAS.

- **Miguel Pérez, J. C. M. (2016). Protección de datos y seguridad de la información (4.a ed.) [Digital]. Ra-Ma Editorial.**
ISBN: 978-84-9964-560-5
- **Cengage Learning. (2017). Hands-On Ethical Hacking and Network Defense, 3rd Edition [Diapositivas; Digital]. Canvas.**